Mobile devices resilient to capture

(MacKenzie & Reiter 2003)

Some statistics (@2010)

➔ **1 laptop stolen every 12 seconds**

➔ **Survey on 329 organizations:**

- ⇨ *86400 laptop lost; value: 2.1 B$*
  - →Data value is greater than PC value!
- ⇨ *43% stolen at home*

➔ **FBI data (2001-2005)**

- ⇨ *160 stolen laptop*
- ⇨ *10+ with critical (classified) data*
- ⇨ *51: unsure whether contained classified data*

➔ **Famous R&B singer, Ryan Leslie**

- ⇨ *Offered 1M$ for his stolen laptop in 2010*
- ⇨ *New original songs and videos worth A LOT for him*
  - →(did not get it back)

Capture resilient device

➔ **A device that CANNOT be used by other than the rightful owner**

➔ **Assume "core" of the device is a secret key**

- ⇨ *E.g. to permit decryption, digital signatures, etc*
- ⇨ *E.g. SIM card*

➔ **Possible security approaches**

- ⇨ *Lock/Unlock key via passwd (e.g. cipher/decipher it)*
  - →Weak: dictionary attack likely to succeed
- ⇨ *Store secret in tamper-proof HW box*
  - →Must have it! Must be robust to side channel analysis
- ⇨ *Dynamically download key from network repository*
  - →Must trust network repository!

➔ **Other ideas?**

MacKenzie + Reiter, 2003

- **Assumption: device is connected when used**
- **Solution: involve a "capture-protection server" in the network**
  - *Server confirms that device remains in owner's possession before permitting usage of key*
  - *SW only, no tamper-proof requirements*
- **"capture-protection server" does NOT need to be trusted!**
- **Two approaches:**
  - *Basic one, standard protocol*
  - *Extended one, uses (2,2) secret sharing*
    - Even if attacker cracks the device (and the user password), device key can be disabled

Scenario
(MacKenzie & Reiter)

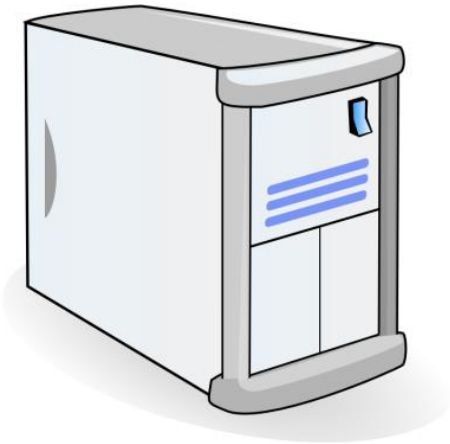


Password P

Many possible attack scenarios
- **password known**
- **device stolen**
- **server cracked (no trusted server!)**

Good: Solution resilient to any crack
Better: Solution resilient to more than 1 crack!





Secret Key *SKd*
Public Key *PKd*

Secret Key *SKs*
Public Key *PKs*

Basic solution

- **Robust to following attacks:**
  - *Server cracked AND password known*
    - Attacker cannot sign/decrypt
  - *Device cracked/stolen*
    - Attacker can only perform dictionary attack ONLINE
  - *Device and Server cracked*
    - Attacker can only perform dictionary attack OFFLINE

- **Basic solution NOT robust to:**
  - *Device AND password cracked*
    - Game over… Attacker can do all

Idea: use “tickets”

➔ **Assumption:**

⇨ *device has access to network*

➔ **Protection:**

⇨ *Encrypt device key so that it can be decrypted only via cooperation with server*

➔ **Idea:**

⇨ *Send encrypted “ticket” to server*

→Contains authentication material for user!!

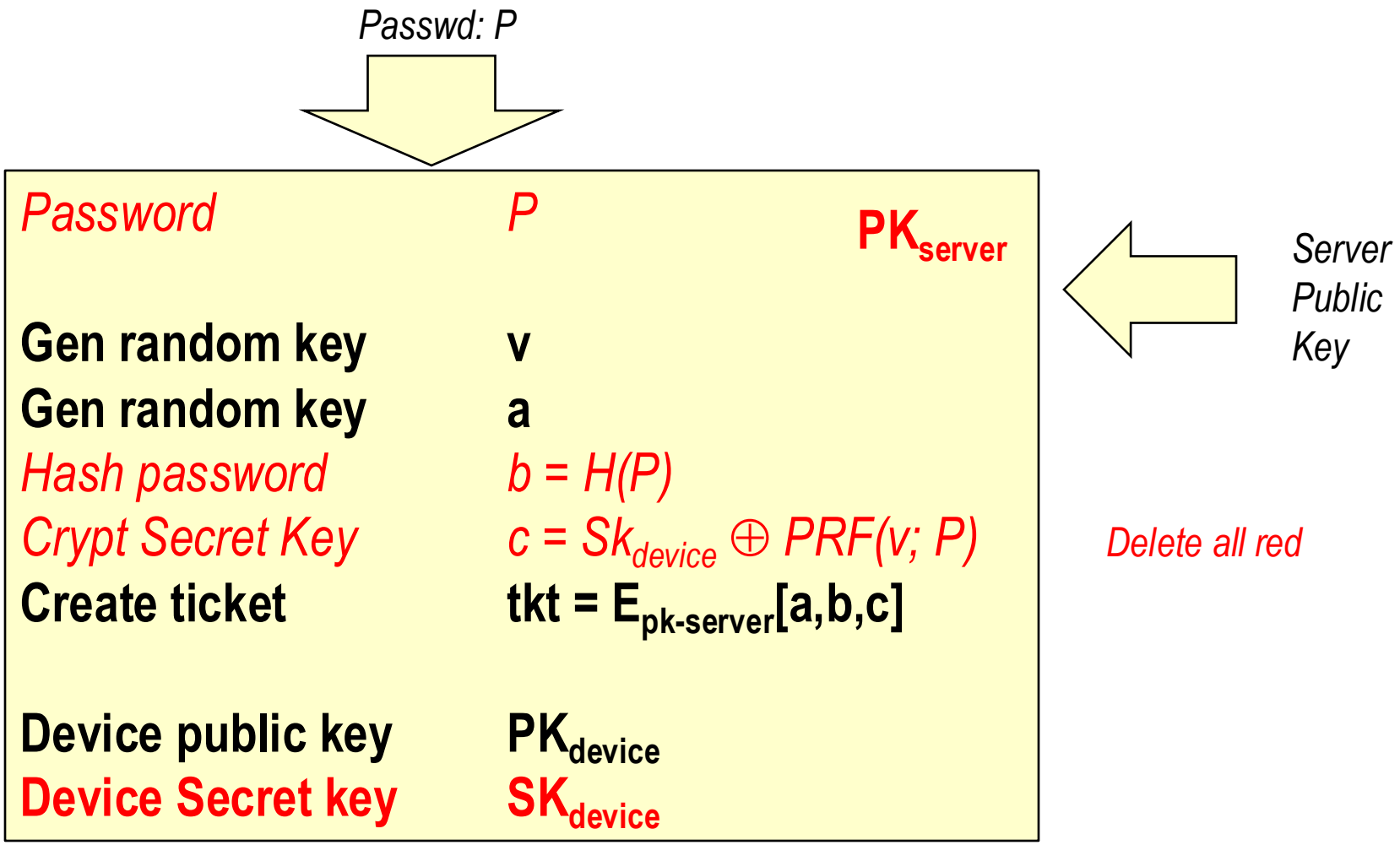
→No need to store on server

⇨ *Use content of ticket to authenticate user*

⇨ *Use content of ticket to “partially” decrypt device key*

→Final decryption at User – server never sees key!

Protocol: device initialization



Protocol: key retrieval

Device

random key	v
random key	a
ticket	tkt = $E_{pk\text{-}server}[a,b,c]$
Device public key	PK_{device}

b=H(P)

c = $Sk_{device} \oplus PRF(v; P)$

User input password P

Compute hash $\beta=H(P)$

Gen random ρ

send $mac_a[tkt , E_{pk\text{-}server}[\beta, \rho]]$ →

Server

tkt → a,b,c

a → check MAC (auth device!)

decrypt → β, ρ

Check → b = β (auth user!)

← $\rho \oplus c$ return

Decrypt key

$(\rho \oplus c) \oplus \rho \oplus PRF[v,P) \rightarrow Sk_{device}$

Attacks

➔ **Server cracked AND password known**

- ⇨ *Key v in device, still secret*
  - →SK encrypted with PFR(v,passwd)
  - →SK cannot be obtained

➔ **Device cracked/stolen**

- ⇨ *Attacker must send valid passwd hash*
- ⇨ *can only perform dictionary attack ONLINE*
  - →Easy to detect!
  - →MAC verified (attacker knows a), but passwd fails many times

➔ **Device and Server cracked**

- ⇨ *Passwd still missing*
- ⇨ *Dictionary attack OFFLINE against b=H(Passwd)*

Can we do better?

- **Limitation**
  - *device stolen and passwd known*
    - Attacket can get SK and use it from now on
- **Solution:**
  - *Must NOT reveal SK to the device itself!*
  - *Easy (now ☺): use secret sharing!*

- **Following example:**
  - *RSA signature*
  - *Analogous for encryption*

Secret sharing (2,2) for RSA

$$n = p \cdot q$$

$$d \leftarrow \text{random, secret key}$$

$$d_1 \leftarrow \text{random, share 1}$$

$$d_2 = d - d_1 \bmod \phi(n) \leftarrow \text{share 2}$$

$$\mathrm{H}(\mathrm{m})^{d_1} \cdot \mathrm{H}(\mathrm{m})^{d_2} = \mathrm{H}(\mathrm{m})^{d_1+d_2} = \mathrm{H}(\mathrm{m})^{d_1+d-d_1} = \mathrm{H}(\mathrm{m})^{d} \bmod n$$

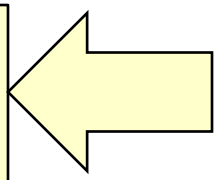
Case (2,2): use trivial secret share

NO Shoup's problem to overcome in reconstruction (no Lagrange now!!)

Protocol2: device initialization

Password	*P*
Gen random key	v
Gen random key	a
Gen share 1	d_1=*PRF(v; P)*
Gen share 2	d_2=d-d_1 mod ϕ(n)
Gen disabling key	t
Gen disabling val	u=H(t)
Hash password	*b = H(P)*
~~*Crypt Secret Key*~~	~~*c = Sk_{device} $\oplus$ PRF(v; P)*~~
Create ticket	tkt = $E_{pk\text{-}server}$[a,b,u,d_2,N]
Device public RSA key (N,e)	
Device Secret RSA key d	[and ϕ(N)]

PK_{server}



Server Public Key

Delete all red & orange

Protocol2: key retrieval

Device – while signing m

random key	v
random key	a
disabling key	t
ticket	tkt = $E_{pk\text{-}server}$[a,b,u,d_2,N]
Device public key	(e,N)

b=H(P)

d_1 = PRF(v; P)

User input password P

Compute hash β=H(P)

Gen random ρ

send → mac_a[tkt , $E_{pk\text{-}server}$[H[m], β, ρ]]

Server

tkt → a,b,d_2,u,N

a → check MAC (auth device!)

decrypt → H[m], β, ρ

Check → b = β (auth user!)

return ← ρ ⊕ $H[m]^{d2}$

Decrypt key

$(\rho \oplus H[m]^{d2}) \oplus \rho$

Complete signature

d_1 = PRF[v,P]

$H[m]^{d2}$ $H[m]^{d1}$ mod N

Signature key d NEVER retrieved in clear!

If attacker gets passwd AND device, cannot get key d

Protocol2: key disabling

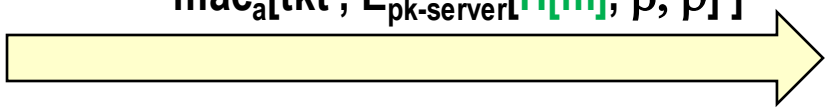
➔**Suffices to keep backup of**

⇨*t*

⇨*tkt*

➔**If device stolen, send them to server**

⇨*Server keeps blacklist*



Server

tkt → a,b,d_2,u,N

a → check MAC

(auth device!)

Search t for index tkt (e.g. hash table)

check u ≠ H(t) for corresponding t

(check if not revoked!)